

تمرین سوم

درس امنیت شبکه های کامپیوتری

نام و نام خانوادگی: محمد امانلو

شماره دانشجویی: ۸۱۰۱۰۰۰۸۴

۱ حل سوال ۱

فرض می کنیم بیت ها مستقل از هم تولید می شوند. پس احتمال هر جفت از ضرب احتمال دو بیتش به دست می آید.

$$P(0) = 0.5 - \delta$$

$$P(1) = 0.5 + \delta$$

الف
احتمال هر جفت در دنباله اصلی .

$$P(00) = P(0)P(0) = (0.5 - \delta)^2$$

$$P(01) = P(0)P(1) = (0.5 - \delta)(0.5 + \delta) = 0.25 - \delta^2$$

$$P(10) = P(1)P(0) = (0.5 + \delta)(0.5 - \delta) = 0.25 - \delta^2$$

$$P(11) = P(1)P(1) = (0.5 + \delta)^2$$

پس نکته مهم اینه که احتمال دو حالت ۰۱ و ۱۰ با هم برابر است.

ب
در الگوریتم اصلاح شده جفت های ۰۰ و ۱۱ حذف می شوند. فقط ۰۱ و ۱۰ باقی می مانند.
طبق صورت سوال داریم.

$$01 \rightarrow 0$$

$$10 \rightarrow 1$$

پس احتمال صفر شدن خروجی برابر است با احتمال ۰۱ به شرط اینکه یکی از دو حالت ۰۱ یا ۱۰ اتفاق افتاده باشد.

$$P_{\text{out}}(0) = \frac{P(01)}{P(01) + P(10)}$$

$$P(01) = P(10) = 0.25 - \delta^2$$

$$P_{\text{out}}(0) = \frac{0.25 - \delta^2}{(0.25 - \delta^2) + (0.25 - \delta^2)} = \frac{1}{2}$$

$$P_{\text{out}}(1) = \frac{P(10)}{P(01) + P(10)}$$

$$P_{\text{out}}(1) = \frac{0.25 - \delta^2}{(0.25 - \delta^2) + (0.25 - \delta^2)} = \frac{1}{2}$$

پس خروجی دیگه سوگیری ندارد و احتمال صفر و یک در خروجی برابر است.

$$P_{\text{out}}(0) = P_{\text{out}}(1) = 0.5$$

ج

هر جفت ورودی فقط وقتی برای ما خروجی تولید می کند که ۰۱ یا ۱۰ باشد. پس احتمال اینکه یک جفت ورودی به یک بیت خروجی تبدیل شود برابر است با.

$$q = P(01) + P(10)$$

$$q = (0.25 - \delta^2) + (0.25 - \delta^2)$$

$$q = 0.5 - 2\delta^2$$

یعنی به طور متوسط از هر جفت ورودی با احتمال q یک بیت خروجی می گیریم.
پس برای تولید یک بیت خروجی به طور متوسط به این تعداد جفت ورودی نیاز داریم.

$$\frac{1}{q}$$

چون هر جفت شامل ۲ بیت است. برای تولید یک بیت خروجی به طور متوسط به این تعداد بیت ورودی نیاز داریم.

$$\frac{2}{q}$$

حالا اگر بخواهیم x بیت خروجی تولید کنیم. تعداد متوسط بیت های ورودی برابر می شود با.

$$x \times \frac{2}{q}$$

$$E[\text{bits input}] = \frac{2x}{0.5 - 2\delta^2}$$

$$E[\text{bits input}] = \frac{4x}{1 - 4\delta^2}$$

$$\boxed{\frac{4x}{1 - 4\delta^2}}$$

۲ حل سوال ۲

نه. این روش امن نیست.

دلیل اصلی اینه که اینجا از RSA به صورت خام و بدون تصادفی سازی استفاده شده. یعنی برای یک پیام مشخص m . خروجی همیشه یک مقدار ثابت C می شود.

$$C = m^e \bmod n$$

از طرفی فضای پیام خیلی کوچک است. چون هر حرف فقط یکی از عددهای ۰ تا ۲۵ است.

$$m \in \{0, 1, 2, \dots, 25\}$$

کلید عمومی باب هم در دسترس همه است.

$$KU = \{e, n\}$$

یعنی مهاجم لازم نیست n را فاکتور بگیرد یا کلید خصوصی را پیدا کند. همین که کلید عمومی را داشته باشد کافی است که برای همه حالت های ممکن جدول بسازد.

$$C_0 = 0^e \bmod n$$

$$C_1 = 1^e \bmod n$$

$$C_2 = 2^e \bmod n$$

$\vdots$

$$C_{25} = 25^e \bmod n$$

مهاجم یک جدول مثل این می سازد.

m	C_m
0	$0^e \bmod n$
1	$1^e \bmod n$
2	$2^e \bmod n$
$\vdots$	$\vdots$
25	$25^e \bmod n$

حالا اگر مهاجم متن رمز شده را در مسیر ببیند. هر عدد رمز شده را با جدول خودش مقایسه می کند. هر جا برابر بود. همان حرف اصلی را پیدا می کند.

$$C = C_m$$

$$m = \text{حرف اصلی}$$

$$|\mathcal{M}| = 26$$

$$\text{Cost} = 26 \text{exponentiations modular}$$

یعنی حتی اگر n خیلی بزرگ باشد و فاکتورگیری از آن عملاً ممکن نباشد. باز هم این روش می شکند. چون مشکل از فاکتورگیری نیست. مشکل از کوچک بودن فضای پیام و قطعی بودن رمزنگاری است. یک مشکل دیگر هم این است که حروف تکراری همیشه به یک رمز تکراری تبدیل می شوند. مثلاً اگر دو بار حرف A فرستاده شود. هر دو بار همان مقدار رمز شده تولید می شود.

$$m_1 = m_2$$

$$C_1 = C_2$$

این باعث می شود الگوهای متن هم لو برود و کار مهاجم راحت تر شود.

۳ حل سوال ۳

روزها را از دوشنبه شماره گذاری می کنیم.

$$0 = \text{دوشنبه}$$

$$1 = \text{سه شنبه}$$

$$2 = \text{چهارشنبه}$$

$$3 = \text{پنجشنبه}$$

$$4 = \text{جمعه}$$

$$5 = \text{شنبه}$$

$$6 = \text{یکشنبه}$$

فرض می کنیم x شماره روزی باشد که دنبال آن هستیم. چون دانشگاه روز یکشنبه اجازه تدریس نمی دهد. باید روز مورد نظریکشنبه باشد.

$$x \equiv 6 \pmod{7}$$

از طرف دیگر برنامه استادها هم به صورت هم نهشتی نوشته می شود. برای استاد اول داریم.

$$x \equiv 0 \pmod{3}$$

برای استاد دوم.

$$x \equiv 1 \pmod{2}$$

برای استاد سوم.

$$x \equiv 2 \pmod{5}$$

برای استاد چهارم.

$$x \equiv 3 \pmod{6}$$

برای استاد با فاصله ۴ روزه که از شنبه شروع کرده است.

$$x \equiv 5 \pmod{4}$$

$$x \equiv 1 \pmod{4}$$

حالا چون شرط $x \equiv 1 \pmod{4}$ خودش فرد بودن x را هم می دهد. شرط استاد دوم هم داخل همین می آید. همچنین ترکیب $x \equiv 0 \pmod{3}$ و فرد بودن x . همان شرط $x \equiv 3 \pmod{6}$ را برای استاد چهارم می دهد. پس دستگاه را به صورت خلاصه اینطوری حل می کنیم.

$$x \equiv 0 \pmod{3}$$

$$x \equiv 1 \pmod{4}$$

$$x \equiv 2 \pmod{5}$$

$$x \equiv 6 \pmod{7}$$

حالا از قضیه باقیمانده چینی استفاده می کنیم.

$$M = 3 \times 4 \times 5 \times 7 = 420$$

برای پیمانه ۳ داریم.

$$a_1 = 0$$

پس سهم این قسمت صفر می شود.
برای پیمانه ۴ داریم.

$$M_2 = \frac{420}{4} = 105$$

$$105 \equiv 1 \pmod{4}$$

$$y_2 = 1$$

$$a_2 M_2 y_2 = 1 \times 105 \times 1 = 105$$

برای پیمانه ۵ داریم.

$$M_3 = \frac{420}{5} = 84$$

$$84 \equiv 4 \pmod{5}$$

$$4y_3 \equiv 1 \pmod{5}$$

$$y_3 = 4$$

$$a_3 M_3 y_3 = 2 \times 84 \times 4 = 672$$

برای پیمانه ۷ داریم.

$$M_4 = \frac{420}{7} = 60$$

$$60 \equiv 4 \pmod{7}$$

$$4y_4 \equiv 1 \pmod{7}$$

$$y_4 = 2$$

$$a_4 M_4 y_4 = 6 \times 60 \times 2 = 720$$

جواب کلی برابر است با.

$$x \equiv 0 + 105 + 672 + 720 \pmod{420}$$

$$x \equiv 1497 \pmod{420}$$

$$1497 = 3 \times 420 + 237$$

$$x \equiv 237 \pmod{420}$$

پس اولین زمانی که همه استادها به حذف جلسه می خورند روز 237 است.

$$\boxed{x = 237}$$

۴ حل سوال ۴

طبق صورت سوال. دو طرف یعنی A و B از قبل کلید عمومی همدیگر را به شکل امن دارند. پس مشکل اصلی ما این نیست که کلید عمومی جعلی به جای کلید اصلی جا زده شود. حالا باید یک پروتکل بسازیم که هم کلید session مشترک تولید کند. هم جلوی حمله Man-in-the-Middle و حمله بازپخش را بگیرد. برای این کار از دینی هلمن همراه با امضای دیجیتال و نانس استفاده می کنیم. پارامترهای عمومی دینی هلمن را اینطوری در نظر می گیریم.

$$q$$

$$\alpha$$

که q یک عدد اول بزرگ است و α ریشه اولیه پیمانه q است.
مرحله اول
ابتدا A یک نانس تصادفی تولید می کند.

$$N_A$$

بعد یک عدد تصادفی خصوصی انتخاب می کند.

$$x_A < q$$

سپس مقدار عمومی دینی هلمن خودش را حساب می کند.

$$Y_A = \alpha^{x_A} \bmod q$$

حالا A مقدار عمومی دینی هلمن و نانس خودش را با کلید خصوصی خودش امضا می کند.

$$Sig_A = Sign_A(Y_A \parallel N_A)$$

بعد پیام زیر را برای B می فرستد.

$$A \rightarrow B : Y_A, N_A, Sig_A$$

مرحله دوم

وقتی B پیام را می گیرد، اول امضای A را با کلید عمومی A بررسی می کند. اگر امضا درست نباشد، پروتکل متوقف می شود. اگر امضا درست باشد، B مطمئن می شود که این پیام واقعا از طرف A آمده و مقدار Y_A در مسیر عوض نشده است. حالا B یک نانس تصادفی تولید می کند.

$$N_B$$

بعد یک عدد تصادفی خصوصی انتخاب می کند.

$$x_B < q$$

سپس مقدار عمومی خودش را حساب می کند.

$$Y_B = \alpha^{x_B} \bmod q$$

کلید مشترک از دید B اینطوری به دست می آید.

$$K = Y_A^{x_B} \bmod q$$

بعد B مقدارهای مهم دو طرف را امضا می کند.

$$Sig_B = Sign_B(Y_B \parallel N_B \parallel Y_A \parallel N_A)$$

و پیام زیر را برای A می فرستد.

$$B \rightarrow A : Y_B, N_B, Sig_B$$

مرحله سوم

وقتی A پیام را دریافت می کند، امضای B را با کلید عمومی B بررسی می کند. اگر امضا درست نباشد، پروتکل متوقف می شود. اگر امضا درست باشد، A مطمئن می شود که مقدار Y_B واقعا از طرف B آمده و به همان session مربوط است. چون داخل امضای B هم مقدارهای Y_A و N_A آمده اند. حالا A کلید مشترک را حساب می کند.

$$K = Y_B^{x_A} \bmod q$$

دو طرف در نهایت به یک کلید مشترک می رسند. چون داریم.

$$K = Y_A^{x_B} \bmod q$$

$$K = (\alpha^{x_A})^{x_B} \bmod q$$

$$K = \alpha^{x_A x_B} \bmod q$$

$$K = Y_B^{x_A} \bmod q$$

$$K = (\alpha^{x_B})^{x_A} \bmod q$$

$$K = \alpha^{x_B x_A} \bmod q$$

پس هر دو مقدار یکی هستند.

$$\alpha^{x_A x_B} \equiv \alpha^{x_B x_A} \pmod{q}$$

چرا این پروتکل امن است

در برابر حمله Man-in-the-Middle امن است. چون مهاجم اگر بخواهد مقدار Y_A یا Y_B را عوض کند. باید بتواند امضای معتبر برای آن بسازد. ولی کلید خصوصی A و B را ندارد. پس نمی تواند خودش را جای یکی از طرفین جا بزند. همچنین امضاها باعث می شوند مقدارهای دیفی هلمن به هویت طرفین وصل شوند. یعنی B مطمئن می شود Y_A واقعا برای A است و A هم مطمئن می شود Y_B واقعا برای B است.

نانس ها هم برای تازگی پیام هستند. یعنی اگر مهاجم یک پیام قدیمی را دوباره پخش کند. طرف مقابل می تواند بفهمد این پیام مربوط به یک اجرای جدید نیست. چون نانس جدید با نانس داخل پیام قدیمی جور در نمی آید. برای اطمینان بیشتر از اینکه هر دو واقعا به همان کلید مشترک رسیده اند. می توان یک مرحله تایید کلید هم اضافه کرد. مثلا دو طرف روی نانس های همدیگر با کلید K یک مقدار MAC حساب کنند.

$$MAC_K(N_A \parallel N_B)$$

۵ حل سوال ۵

الف

در این سیستم برای هر کاربر به جای خود رمز عبور. مقدار هش شده رمز ذخیره شده است.

$$H = h(p)$$

که p همان رمز عبور کاربر است. موقع ورود هم سیستم رمز وارد شده را هش می کند و با مقدار ذخیره شده مقایسه می کند.

$$h(p') = H$$

اگر برابر بود. ورود انجام می شود.

حالا فرض کنیم مهاجم فقط لیست هش رمزها را دارد. یعنی برای هر کاربر مقدار H را می بیند. ولی خود رمز را نمی داند.

حمله A. شکستن خاصیت یک طرفه بودن

در این حمله مهاجم از روی مقدار هش شده دنبال ورودی آن می گردد.

$$H = h(p)$$

یعنی می خواهد یک p' پیدا کند که داشته باشیم.

$$h(p') = H$$

اگر مهاجم بتواند این کار را انجام دهد. می تواند با همان p' وارد حساب کاربر شود. حتی لازم نیست حتما p' دقیقا همان رمز اصلی باشد. همین که هش آن با هش ذخیره شده برابر شود کافی است. پس این حمله برای ورود به حساب کاربران جواب می دهد.

حمله A خطرناک است و امکان جعل هویت می دهد

حمله B. پیدا کردن تصویر ثانویه

در حمله تصویر ثانویه معمولا یک ورودی مشخص p را داریم و می خواهیم یک ورودی دیگر p' پیدا کنیم که.

$$p' \neq p$$

$$h(p') = h(p)$$

ولی در این سوال مهاجم خود رمز اصلی p را ندارد. فقط مقدار هش شده را دارد. پس عملاً نمی تواند حمله تصویر ثانویه را به شکل معمولش اجرا کند. از طرفی اگر مهاجم خود رمز اصلی را داشته باشد. دیگر نیازی به پیدا کردن تصویر ثانویه ندارد. چون با همان رمز اصلی می تواند وارد شود. پس این حمله در این سناریو به درد مهاجم نمی خورد.

حمله B به تنهایی امکان ورود به حساب کاربران را نمی دهد

حمله C. پیدا کردن تصادم در حمله تصادم مهاجم فقط دو ورودی متفاوت پیدا می کند که هش یکسان دارند.

$$x \neq y$$

$$h(x) = h(y)$$

اما این دو مقدار لزوماً هیچ ربطی به هش رمز یکی از کاربران ندارند. برای ورود به حساب یک کاربر. مهاجم باید رمزی پیدا کند که هش آن دقیقاً برابر هش ذخیره شده همان کاربر باشد.

$$h(p') = H_{\text{user}}$$

ولی حمله تصادم فقط می گوید دو مقدار تصادفی با هش برابر پیدا شده اند. نه اینکه این هش برابر هش یکی از کاربران باشد. پس این حمله هم برای ورود به حساب کاربران کافی نیست.

حمله C به تنهایی امکان جعل هویت نمی دهد

در نتیجه بین این سه حمله. حمله ای که مستقیماً به مهاجم کمک می کند حمله A است.

حمله A

ب در روش بهتر. قبل از هش کردن رمز عبور یک مقدار تصادفی به اسم Salt به آن اضافه می شود.

$$H = h(s \parallel p)$$

که s همان Salt است و معمولاً کنار هش رمز ذخیره می شود. نکته مهم اینه که Salt لازم نیست مخفی باشد. هدفش این نیست که مثل کلید سری عمل کند. هدفش اینه که کاری کند دو کاربر با رمز عبور یکسان. هش یکسان نداشته باشند. مثلاً اگر دو کاربر رمز یکسان داشته باشند.

$$p_1 = p_2$$

بدون Salt داریم.

$$h(p_1) = h(p_2)$$

ولی با Salt اگر Salt ها فرق داشته باشند.

$$h(s_1 \parallel p_1) \neq h(s_2 \parallel p_2)$$

این کار باعث می شود جدول های آماده و حمله های دیکشنری از قبل ساخته شده خیلی کمتر به درد بخورند. چون مهاجم برای هر Salt باید جداگانه حدس ها را امتحان کند. اثر Salt روی حمله A این است که مهاجم هنوز هم می تواند برای یک کاربر خاص حدس بزند.

$$h(s \parallel p') = H$$

اما دیگر نمی تواند یک جدول آماده برای همه کاربران استفاده کند. برای هر کاربر و هر Salt باید محاسبه جدا انجام دهد. پس حمله سخت تر و کندتر می شود. اثر Salt روی حمله B خیلی مستقیم نیست. چون در این سوال مهاجم اصلا رمز اصلی را ندارد که بخواهد تصویر ثانویه آن را پیدا کند. اثر Salt روی حمله C هم زیاد مهم نیست. چون تصادم های کلی هنوز هم لزوما به هش یک کاربر خاص ربطی ندارند. پس Salt بیشتر جلوی حمله های دیکشنری آماده. جدول های پیش محاسبه شده و Table Rainbow را می گیرد. همچنین باعث می شود اگر چند کاربر رمز یکسان داشته باشند. از روی هش های ذخیره شده این موضوع معلوم نشود.

Salt امنیت ذخیره سازی رمز را بهتر می کند. ولی جایگزین تابع هش قوی نیست

۶ حل سوال ۶

الف

در پروتکل اول پیام ارسالی این است.

$$y = e_{k_1}(x \parallel H(k_2 \parallel x))$$

گیرنده بعد از گرفتن y اول آن را با کلید k_1 رمزگشایی می کند.

$$x \parallel t = d_{k_1}(y)$$

که در آن داریم.

$$t = H(k_2 \parallel x)$$

بعد خودش با استفاده از k_2 مقدار هش را دوباره حساب می کند.

$$t' = H(k_2 \parallel x)$$

حالا دو مقدار را مقایسه می کند.

$$t' = t$$

اگر برابر باشند پیام پذیرفته می شود. اگر برابر نباشند یعنی پیام در مسیر تغییر کرده یا پیام معتبر نیست. در پروتکل دوم پیام ارسالی این است.

$$y = e_k(x \parallel \text{Sig}_{k_{pr}}(H(x)))$$

گیرنده بعد از گرفتن y اول آن را با کلید مشترک k رمزگشایی می کند.

$$x \parallel \text{sig} = d_k(y)$$

بعد هش پیام را حساب می کند.

$$h = H(x)$$

سپس با کلید عمومی فرستنده امضا را بررسی می کند.

$$Verify_{k_{pu}}(h, sig)$$

اگر امضا درست باشد پیام پذیرفته می شود. اگر امضا درست نباشد پیام رد می شود.

ب

بررسی پروتکل اول

در پروتکل اول محرمانگی داریم. چون کل پیام با کلید متقارن k_1 رمز شده است و فقط فرستنده و گیرنده این کلید را دارند.

$$y = e_{k_1}(x \parallel H(k_2 \parallel x))$$

پس کسی که k_1 را ندارد نمی تواند متن پیام x را بخواند. صحت یا یکپارچگی هم داریم. چون مقدار زیر همراه پیام فرستاده شده است.

$$H(k_2 \parallel x)$$

از آنجایی که k_2 محرمانه است. مهاجم نمی تواند بعد از تغییر دادن x یک هش معتبر جدید بسازد. پس اگر پیام تغییر کند گیرنده موقع مقایسه هش ها متوجه می شود.

اما عدم انکار نداریم. چون k_1 و k_2 بین فرستنده و گیرنده مشترک هستند. یعنی گیرنده هم می تواند دقیقا همین نوع پیام را بسازد. پس بعدا نمی شود به یک شخص سوم ثابت کرد که حتما فرستنده این پیام را ساخته است.

پروتکل اول محرمانگی و صحت دارد. ولی عدم انکار ندارد

بررسی پروتکل دوم

در پروتکل دوم محرمانگی داریم. چون پیام با کلید مشترک k رمز شده است.

$$y = e_k(x \parallel Sig_{k_{pr}}(H(x)))$$

پس کسی که کلید k را ندارد نمی تواند پیام اصلی را بخواند. صحت هم داریم. چون روی هش پیام امضای دیجیتال زده شده است.

$$Sig_{k_{pr}}(H(x))$$

اگر مهاجم x را تغییر دهد. مقدار $H(x)$ عوض می شود و امضا دیگر با پیام جدید جور در نمی آید. پس گیرنده می فهمد پیام دستکاری شده است.

عدم انکار هم در این پروتکل داریم. چون امضا با کلید خصوصی فرستنده ساخته شده است.

$$k_{pr}$$

این کلید فقط دست فرستنده است. گیرنده فقط با کلید عمومی فرستنده امضا را بررسی می کند و نمی تواند خودش همان امضا را جعل کند. پس فرستنده بعدا نمی تواند راحت بگوید من این پیام را نفرستاده ام.

پروتکل دوم محرمانگی. صحت. و عدم انکار را تامین می کند